

NanoPoreSeq

NanoPoreSeq — Guide utilisateur

Simuler des BAM Nanopore de tumeurs cérébrales et tester Rapid-CNS2_nf

Version du 26 octobre 2026 (2) · scripts des dossiers simulation/, rapidcns2/ et usb/

Ce projet est développé avec l'aide de l'IA (Claude, Anthropic), sous la relecture active de Pierre Bady.

Sommaire

1. À propos de ce guide
2. Présentation
3. Démarrage rapide
4. Installation
5. Notions clés
6. Utilisation
7. Référence des commandes
8. Référence des fichiers
9. Ressources et durées
10. Dépannage
11. Glossaire
12. Annexe : code de génération de la clé USB

1. À propos de ce guide

Ce guide explique comment installer et utiliser NanoPoreSeq : simuler des BAM Nanopore de gliomes, les vérifier, obtenir leur classe de méthylation, simuler des cohortes, puis s'en servir pour tester le pipeline Rapid-CNS2_nf, y compris depuis une clé USB autonome.

Il s'adresse aux bioinformaticiens et aux utilisateurs de Rapid-CNS2 à l'aise avec la ligne de commande Linux. Le fonctionnement interne et l'historique des corrections sont décrits séparément dans le rapport de relecture (dossier docs/).

Développement

Ce projet est développé avec l'aide de l'IA (Claude, Anthropic), sous la relecture active de Pierre Bady. Le code, les tests et la documentation sont produits avec l'assistant, puis relus, discutés et validés par l'auteur ; les tests rapportés dans ce guide ont été réellement exécutés.

Conventions

- Les commandes sont à taper dans un terminal, depuis le dossier indiqué (simulation/, rapidcns2/ ou usb/).
- $DATA désigne le dossier data/ de Rapid-CNS2_nf et $REF la FASTA hg38 (voir 4.2). Un « \ » en fin de ligne indique que la commande continue sur la ligne suivante.
- Les noms entre chevrons, comme <id>, sont à remplacer par la valeur réelle (nom d'échantillon, dossier…).
- Les durées et la mémoire citées ont été mesurées sur une machine de test de 7 Go de mémoire, avec la séquence de remplacement (voir 5.2).

2. Présentation

NanoPoreSeq produit des fichiers BAM qui imitent un séquençage Nanopore à échantillonnage adaptatif (panel NPHD, génome hg38) d'une tumeur gliale. Chaque BAM contient les altérations connues d'un profil tumoral (variants, nombre de copies, méthylation) ; un fichier de vérité terrain les décrit. On peut ainsi vérifier qu'un pipeline comme Rapid-CNS2_nf retrouve exactement ce qui a été simulé.

Contenu

Dossier	Sert à	Scripts
`simulation/`	simuler, vérifier, classer, simuler des cohortes	`simulate_glioma_hg38.py, verify_bam.py, build_cohort.py, bedmethyl_from_bam.py, methylation_classification_local.R, 01_export_probe_targets.R, 02_export_cohort_probe_targets.R, run_full_mnp_classification.sh, simnanocns_gui_en.py`
`rapidcns2/`	lancer Rapid-CNS2_nf ; préparer des données réelles	`rapidcns2_gui_en.py, align_fastq_rapidcns2_en.sh`
`usb/`	construire une clé USB autonome	`make_usb_stick_en.sh, prepare_usb_en.sh`
`docs/`	documentation	ce guide, rapports de relecture

À noter : Les scripts d'un même dossier s'appellent entre eux : gardez-les ensemble. L'arborescence complète du projet et des dossiers produits est décrite en 8.5.

Parcours type

1. Simuler un ou plusieurs BAM (section 6.1 ou 6.4).
2. Vérifier chaque BAM avec verify_bam.py (6.2).
3. Facultatif : obtenir la classe de méthylation attendue (6.3).
4. Analyser les BAM avec Rapid-CNS2_nf et comparer le rapport à la vérité terrain (6.6).

3. Démarrage rapide

Ce test de quelques minutes vérifie que tout fonctionne, sans référence hg38 : il simule un glioblastome sur le seul chromosome 10 avec la séquence de remplacement, puis le vérifie.

1. Installer les dépendances Python et récupérer les données de Rapid-CNS2_nf.

```
python3 -m pip install pysam numpy
git clone https://github.com/areebapatel/Rapid-CNS2_nf
export DATA=$PWD/Rapid-CNS2_nf/data
```

2. Simuler (environ une minute).

```
cd NanoPoreSeq/simulation
python3 simulate_glioma_hg38.py --profile gbm --standin --data-dir $DATA \
    --out test_gbm --chroms chr10
```

3. Vérifier.

```
python3 verify_bam.py --bam test_gbm/SIM_GBM_HG38.bam --data-dir $DATA --standin
```

4. Lire la fin de la sortie : la méthylation MGMT doit être proche de 0,88, la couverture MGMT supérieure à 10× et le taux de mésappariement proche de 2 %.

> **À noter :** Un BAM simulé avec --standin sert uniquement à tester la chaîne. Pour Rapid-CNS2, simulez avec la vraie référence (--ref).

4. Installation

4.1 Logiciels

Composant	Nécessaire pour	Installation
Python 3, pysam, numpy	tout ce qui est en Python	`python3 -m pip install pysam numpy`
Streamlit ≥ 1,37, pandas	interface de simulation	`python3 -m pip install "streamlit>=1.37" pandas`
tkinter	interface Rapid-CNS2	`sudo apt install python3-tk`
R et paquets	classification, export des profils	voir ci-dessous
Nextflow, Java ≥ 17, Docker / Apptainer	Rapid-CNS2_nf	documentation de Rapid-CNS2_nf
dorado, samtools	données réelles (FASTQ)	site d'Oxford Nanopore ; gestionnaire de paquets
rsync, curl	clé USB	`sudo apt install rsync curl`

> **À noter :** Nextflow 26.04 et plus analyse les fichiers avec une syntaxe stricte que Rapid-CNS2_nf 3.0 ne respecte pas encore (erreur « Unexpected input: 'java' » au démarrage). L'interface Rapid-CNS2 et la clé USB règlent automatiquement NXF_SYNTAX_PARSER=v1 (ancien analyseur) ; en ligne de commande, faire export NXF_SYNTAX_PARSER=v1 avant nextflow run.

Paquets R : optparse, ranger, matrixStats, data.table, glmnet et GenomicRanges. Sous Ubuntu, les paquets du système suffisent (méthode utilisée pour les tests) ; ailleurs, utilisez CRAN et Bioconductor.

```
sudo apt install r-base-core r-cran-optparse r-cran-ranger r-cran-matrixstats \
    r-cran-data.table r-cran-glmnet r-bioc-genomicranges
# ou
Rscript -e
'install.packages(c("optparse","ranger","matrixStats","data.table","glmnet","BiocManager"))'
Rscript -e 'BiocManager::install("GenomicRanges")'
```

4.2 Données et référence

1. Récupérer Rapid-CNS2_nf : son dossier data/ fournit le panel NPHD, la région MGMT, les gènes, refGene et les données d'entraînement de Capper et al.

   ```
   git clone https://github.com/areebapatel/Rapid-CNS2_nf
   export DATA=$PWD/Rapid-CNS2_nf/data
   ```

2. Indiquer une FASTA hg38. Les contigs doivent s'appeler chr1… (UCSC) ou NC_000001.11… (NCBI RefSeq). L'index .fai est créé automatiquement s'il manque et si le dossier est accessible en écriture.

   ```
   export REF=/path/to/hg38.fa
   ```

4.3 Contrôle

Chaque script en ligne de commande affiche ses options et leurs valeurs par défaut avec --help. Le démarrage rapide (section 3) valide l'installation Python ; la section 6.3 valide l'installation R.

```
python3 simulate_glioma_hg38.py --help
Rscript methylation_classification_local.R --help
```

5. Notions clés

5.1 Les trois profils tumoraux

Altération	gbm	astro	oligo
Diagnostic	glioblastome IDH-sauvage	astrocytome IDH-muté	oligodendrogliome IDH-muté, codélété 1p/19q
Chromosomes 7 / 10	gain / perte	normaux	normaux
1p / 19q	—	—	perte d'une copie
EGFR	amplification	—	—
CDKN2A/B	délétion homozygote	option grade 4 (--cdkn2a-del)	—
IDH1 R132H	non	oui	oui
TERT C228T	oui (VAF 0,80)	non	oui
TP53 R175H	—	oui	non
Délétion de 1 pb	—	ATRX	CIC
Promoteur MGMT	méthylé	méthylé	méthylé
Classe de méthylation attendue	GBM_RTK_II	A_IDH (A_IDH_HG en grade 4)	O_IDH

5.2 Séquence réelle ou séquence de remplacement

Avec --ref, les reads sont tirés de la vraie FASTA hg38 : le BAM est utilisable par Rapid-CNS2. Avec --standin, ils sont tirés d'une séquence synthétique aux mêmes coordonnées : c'est plus rapide à mettre en place et suffit pour tester les scripts, mais un tel BAM ne s'aligne pas sur le vrai génome et ne doit pas être donné à Rapid-CNS2.

5.3 Pureté et fréquence allélique

La pureté tumorale (--purity, 0,80 par défaut) est la part de cellules tumorales. Un variant hétérozygote apparaît alors dans une fraction pureté/2 des reads (0,40 par défaut) ; TERT C228T du profil gbm est fixé à 0,80. La fréquence mesurée varie d'un échantillon à l'autre selon la profondeur.

5.4 Vérité terrain

Chaque simulation écrit truth_events.tsv à côté du BAM : la liste exacte des événements simulés (section 8.1). C'est la référence pour vérifier le BAM et pour juger le rapport de Rapid-CNS2.

5.5 Profils de méthylation réels (sondes MNP)

Sans autre option, la méthylation du BAM est un fond générique qui ne permet pas de classification réaliste. Avec --mnp-targets, le simulateur reproduit, aux positions des sondes HM450 utilisées par le classificateur, le profil binarisé d'un vrai échantillon de la classe visée (données de Capper et al.). Les fichiers de profils sont produits par 01_export_probe_targets.R (un représentant par classe) ou 02_export_cohort_probe_targets.R (plusieurs échantillons distincts, pour les cohortes).

5.6 Reproductibilité

Une même graine (--seed) avec les mêmes options et la même version des scripts donne le même BAM. Dans une cohorte, l'individu i reçoit la graine --seed-base + i.

6. Utilisation

6.1 Simuler un échantillon

1. Choisir le profil, la pureté et un nom d'échantillon.
2. Lancer la simulation sur le génome entier (chr1–22 et chrX).

```
python3 simulate_glioma_hg38.py --profile astro --ref $REF --data-dir $DATA \
    --out sim_astro --sample-id ASTRO01 --purity 0.7
```

3. Le script contrôle d'abord la base hg38 de chaque point chaud, affiche le nombre de reads par chromosome, puis « Done ». Le BAM et son index sont dans le dossier --out, avec truth_events.tsv.

Variantes utiles :

- Astrocytome de grade 4 : ajouter --cdkn2a-del.
- Classification réaliste : ajouter --mnp-targets probe_targets/probe_targets_astro.tsv (voir 6.3).
- Variabilité des CNV : --cnv-jitter-sd 0.15 (environ ±15 %).
- Test rapide : --chroms chr2,chr19 limite la simulation à quelques chromosomes.
- Couverture : --ontarget-depth (panel), --offtarget-depth (hors panel, sert aux CNV), --mgmt-depth-factor (promoteur MGMT).

6.2 Vérifier un échantillon

```
python3 verify_bam.py --bam sim_astro/ASTRO01.bam --data-dir $DATA --ref $REF
```

Avec un BAM simulé par --standin, remplacer --ref $REF par --standin et donner la même --seed que la simulation. La sortie est découpée en sections :

Section	Ce qui est mesuré	Résultat attendu
`Header / format`	modèle de basecalling, contigs, reads avec tag MM	25 contigs ; presque tous les reads ont un tag MM
`Mean coverage in the NPHD panel`	couverture du panel par chromosome	indicatif seulement
`Copy number`	rapport de copies des grands événements	proche de la valeur « expected » (0,50 pour une perte, 1,5 pour un gain)
`Focal events`	EGFR, CDKN2A/B	EGFR nettement au-dessus du panel, CDKN2A/B presque nul
`Hotspots`	comptes de bases et VAF	VAF proche de pureté/2 pour les variants ; quelques % pour les sites sauvages (erreurs)
`MGMT`	profondeur et méthylation du promoteur ; témoin	≥ 10× ; ~0,88 sur les deux brins ; témoin proche du fond attendu
`Alignment / sequence`	taux de mésappariement	~2 %

> **À noter :** Avec --chroms, les événements situés sur des chromosomes non simulés sont signalés « not simulated » : c'est normal.

6.3 Obtenir la classe de méthylation

La méthode la plus simple enchaîne les quatre étapes (export du profil, simulation avec sondes MNP, bedMethyl, classification) pour un profil :

```
./run_full_mnp_classification.sh --profile astro --data-dir $DATA --ref $REF \
    --num-trees 20000 --threads 16
```

Le résultat s'affiche à la fin et se trouve dans classif_astro/. Pour classer un BAM déjà simulé avec --mnp-targets :

```
python3 bedmethyl_from_bam.py --bam sim/X.bam --out sim/X.bedmethyl.bed
Rscript methylation_classification_local.R -s X -o classif_X -i sim/X.bedmethyl.bed \
    -p $DATA/top_probes_hm450.Rdata -a $DATA/HM450.hg38.manifest.gencode.v22.Rdata \
    -d $DATA/capper_top_100k_betas_binarised.Rdata -t 16 -n 20000
```

- Lire classif_X/X_calibrated_classification.tsv : « Predicted Class » doit être la classe attendue du profil (section 5.1).
- Nombre d'arbres : 20 000 comme en production ; 2 000 pour un criblage ; moins pour un simple test. La mémoire augmente avec le nombre d'arbres (section 9).
- Des avertissements « glm.fit: algorithm did not converge » sont normaux (section 10).

6.4 Simuler une cohorte

1. Exporter les profils réels : n_per_class = effectif du plus grand groupe (33 pour 99 individus en parts égales).

   ```
   Rscript 02_export_cohort_probe_targets.R $DATA cohort_probes 33
   ```

2. Faire un essai réduit (quelques individus, un chromosome).

   ```
   python3 build_cohort.py --n 3 --standin --data-dir $DATA --out-dir cohort_test \
       --chroms chr17 --jobs 3 --mnp-targets-dir cohort_probes
   ```

3. Lancer la cohorte complète, avec ou sans classification.

```
python3 build_cohort.py --n 99 --ref $REF --data-dir $DATA --out-dir cohort_bams \
    --mnp-targets-dir cohort_probes --jobs 8 --cnv-jitter-sd 0.15 \
    --astro-grade4-fraction 0.3 --classify --classify-jobs 2 --num-trees 2000
```

4. Contrôler cohort_bams/cohort_manifest.tsv (toutes les lignes au statut OK) et, avec --classify, le taux de classification correcte affiché à la fin et le résumé cohort_classification/cohort_classification_summary.tsv.
   - Répartition : parts égales par défaut ; --profile-counts gbm=50,astro=30,oligo=19 impose des effectifs (somme = --n).
   - Pureté tirée au hasard dans --purity-range (0,60 à 0,95 par défaut).
   - --jobs règle le nombre de simulations simultanées et --classify-jobs celui des classifications : adaptez-les à la mémoire (section 9).
   - La classe A_IDH_HG ne compte que 46 échantillons réels : au-delà, des profils sont réutilisés (un avertissement l'indique).

6.5 Interface de simulation

1. Démarrer l'interface sur la machine de calcul. Depuis un autre poste, ouvrir un tunnel SSH puis http://localhost:8501.

```
streamlit run simnanocns_gui_en.py
ssh -L 8501:localhost:8501 <machine>
```

2. Barre latérale : dossier de travail, dossier des scripts (le dossier simulation/), interpréteur Python, dossier data/, référence (hg38 réelle ou séquence de remplacement). « Save as defaults » mémorise ces réglages ; « Export / Import (JSON) » permet de les partager.
3. Onglet Single sample (un échantillon) ou Cohort (cohorte) : régler les paramètres. La colonne de droite affiche la commande exacte et les contrôles ; le bouton de lancement reste grisé tant qu'une erreur subsiste.
4. Onglet Run : durée, nombre de BAM produits, journal en direct, puis résultats (vérité terrain, manifeste, taux de classification). Les journaux sont conservés dans <dossier de travail>/gui_logs/.

> **À noter :** Le calcul continue si l'onglet du navigateur est fermé, mais s'arrête si le serveur Streamlit est arrêté.

6.6 Tester Rapid-CNS2 avec les BAM simulés

1. Simuler avec --ref (jamais --standin) et, pour une classification interprétable, avec --mnp-targets.
2. Données simulées : l'interface coche d'elle-même la case « Simulated BAM » quand l'en-tête du BAM porte la mention « SIMULATED DATA » écrite par le simulateur. Elle écrit alors .rapidcns2_simulated.config dans le dossier de lancement et l'ajoute à la commande : les échecs d'AnnotSV (aucun variant de structure, donc pas de table) et de SAVANA CNA sont ignorés, sans effet sur le rapport. Décocher la case pour des données réelles. En ligne de commande, le même effet s'obtient avec ce fichier passé par -c :

```
process {
    withName: 'annotSv.*' { errorStrategy = 'ignore' }
    withName: 'savanaCna' { errorStrategy = 'ignore' }
}
```

3. Ouvrir l'interface Rapid-CNS2, choisir le dossier de cohorte et l'échantillon, vérifier les chemins et les options, puis « Run ». « Copy command » donne la commande équivalente pour un terminal. L'interface monte automatiquement les dossiers AnnotSV et ANNOVAR dans les conteneurs (--containerBindPaths) ; RCNS_BINDPATHS permet d'en ajouter d'autres, séparés par des virgules.

```
cd NanoPoreSeq/rapidcns2
python3 rapidcns2_gui_en.py
```

4. À la fin, « Open report » ouvre le rapport HTML. Le comparer à truth_events.tsv à l'aide du tableau ci-dessous.

Élément du rapport	gbm	astro	oligo
Classe de méthylation	GBM_RTK_II	A_IDH (A_IDH_HG si grade 4)	O_IDH
Statut MGMT	méthylé	méthylé	méthylé
Profil de nombre de copies	+7, −10, amplification EGFR, délétion CDKN2A/B	plat (délétion CDKN2A/B si grade 4)	perte 1p et 19q
Variants	TERT C228T	IDH1 R132H, TP53 R175H	IDH1 R132H, TERT C228T

> **À noter :** Vérifier aussi mgmt/mgmt_avg_cov.txt : si la couverture MGMT est sous --minimumMgmtCov (10 dans l'interface), le module MGMT est sauté.

6.7 Préparer des données réelles

Pour des FASTQ issus d'un vrai séquençage, align_fastq_rapidcns2_en.sh fusionne les fichiers, aligne sur hg38, trie, indexe et contrôle le résultat.

```
./align_fastq_rapidcns2_en.sh -i fastq_pass/ -r $REF -s GBM01 -o results -t 24
```

> **À noter :** La méthylation n'est disponible que si les FASTQ portent les tags MM/ML dans leurs en-têtes (par exemple produits par samtools fastq -T '*'). Un FASTQ MinKNOW standard n'en contient pas : repartir alors des POD5 ou du BAM non aligné.

6.8 Clé USB autonome

La méthode recommandée est make_usb_stick_en.sh : il contrôle que le périphérique est bien une clé amovible (jamais un disque système), l'efface et la formate en ext4 après confirmation, la monte, puis lance prepare_usb_en.sh qui copie le pipeline, Nextflow, Java, les conteneurs, la référence, ANNOVAR, AnnotSV, l'interface et les BAM demandés, et termine par run.sh --check. Lancez-le avec votre compte habituel : il appelle sudo lui-même pour les étapes privilégiées.

1. Brancher la clé et repérer son nom (colonnes TRAN = usb ou RM = 1).

```
cd NanoPoreSeq/usb
./make_usb_stick_en.sh --list
```

2. Simuler l'opération sans rien modifier, puis vérifier la place nécessaire (environ 100 Go ou plus selon les conteneurs et ANNOVAR).

```
./make_usb_stick_en.sh --dry-run /dev/sdX
```

3. Créer la clé. Le script demande de taper le nom du périphérique pour confirmer l'effacement ; les options placées après « -- » sont transmises à prepare_usb_en.sh. Les chemins sources se règlent par les variables PIPELINE_DIR, REF_FA, ANNOVAR_DIR et ANNOTSV_DIR.

```
./make_usb_stick_en.sh --eject /dev/sdX -- --engine apptainer \
    --with-samples /path/to/cohort_bams
```

4. Sur l'ordinateur hôte : contrôler, puis lancer l'interface ou la ligne de commande. Le dossier de travail de Nextflow va sur le disque de l'hôte (prévoir ~150 Go par échantillon ; modifiable par RCNS_WORKDIR).

```
cd /media/$USER/RAPIDCNS2/rapidcns2
./run.sh --check
./run_gui.sh
./run.sh samples/IND001_GBM/IND001_GBM.bam IND001_GBM
```

Option de make_usb_stick_en.sh	Effet
/dev/sdX	disque entier : nouvelle table de partitions GPT, une seule partition ext4
/dev/sdX1	partition : seule cette partition est reformatée
/media/$USER/RAPIDCNS2	dossier de montage d'une clé déjà formatée : rien n'est effacé (comme --keep-format)
--keep-format	garde le système de fichiers ext4 existant et complète la clé (mise à jour)
--label NAME	étiquette du système de fichiers (RAPIDCNS2 par défaut)
--eject	démonte et éteint la clé à la fin
--yes	sans confirmation interactive (les contrôles de sécurité restent actifs)
--allow-non-removable	accepte un disque non signalé comme amovible/USB (certains boîtiers SSD)
--mountpoint DIR	point de montage si udisksctl est absent
--dry-run, --list	simulation sans modification ; liste des disques

> **À noter :** Sécurité : le script refuse tout disque dont une partition est montée ailleurs que sous /media, /run/media, /mnt (ou le dossier --mountpoint) ou utilisée comme swap, ce qui exclut le disque système, même avec --allow-non-removable. Tout le contenu de la clé est effacé, sauf avec --keep-format.

Méthode manuelle, équivalente : formater la clé soi-même (lsblk, umount, mkfs.ext4 -m 0 -L RAPIDCNS2, rebrancher, chown), puis lancer prepare_usb_en.sh sur son point de montage (section 12.3).

- --no-annovar : ANNOVAR n'est pas copié (il ne peut pas être redistribué) ; sur l'hôte, pointer RCNS_ANNOVARPATH et RCNS_ANNOVARDB vers une installation locale.
- Hors ligne, wf-human-variation est désactivé (RCNS_HUMANVAR=true pour l'activer avec un accès réseau) et l'envoi automatique vers MNP-Flex est impossible.
- conf/usb.config ignore les échecs d'AnnotSV et de SAVANA : commenter ces lignes pour des données réelles.
- Le code complet des deux scripts de la clé figure en annexe, section 12.

7. Référence des commandes

7.1 simulate_glioma_hg38.py

Option	Défaut	Description
--profile	obligatoire	gbm, astro ou oligo
--ref FASTA	—	FASTA hg38 réelle (un des deux, --ref ou --standin)
--standin	—	séquence de remplacement (tests)
--data-dir	obligatoire	dossier data/ de Rapid-CNS2_nf

Option	Défaut	Description
`--out`	sim_glioma	dossier de sortie
`--sample-id`	SIM_<PROFILE>_HG38	nom de l'échantillon et du BAM
`--purity`	0,80	pureté tumorale, entre 0 et 1
`--ontarget-depth`	8	profondeur par région du panel (couverture effective ~2,5 fois plus haute)
`--offtarget-depth`	2	profondeur hors panel
`--mgmt-depth-factor`	4	profondeur MGMT = facteur × --ontarget-depth × rapport de copies local
`--cdkn2a-del`	—	astro : délétion homozygote CDKN2A/B (grade 4)
`--cnv-jitter-sd`	0	écart-type du bruit sur les rapports de copies
`--chroms`	chr1–22, chrX	liste séparée par des virgules
`--seed`	2026	graine aléatoire
`--mnp-targets`	—	fichier de profil réel aux sondes HM450
`--mnp-depth`	6	profondeur dédiée par sonde
`--mnp-window`	150	demi-fenêtre autour de chaque sonde (pb)
`--skip-ref-check`	—	saute le contrôle des bases de référence

7.2 verify_bam.py

Option	Défaut	Description
`--bam`	obligatoire	BAM simulé
`--data-dir`	obligatoire	dossier data/ de Rapid-CNS2_nf
`--ref / --standin`	—	comme pour la simulation
`--truth`	à côté du BAM	fichier truth_events.tsv
`--seed`	2026	graine de la simulation (avec --standin)

7.3 build_cohort.py

Option	Défaut	Description
`--n`	99	nombre d'individus
`--profile-counts`	parts égales	effectifs, ex. gbm=50,astro=30,oligo=19
`--ref / --standin, --data-dir`	—	comme pour la simulation
`--out-dir`	cohort_bams	dossier de sortie
`--mnp-targets-dir`	—	profils réels (sortie de 02_export…) ; requis avec --classify et --astro-grade4-fraction
`--mnp-depth`	6	profondeur par sonde
`--chroms`	chr1–22, chrX	restreint les chromosomes
`--purity-range MIN MAX`	0,60 0,95	intervalle de pureté

Option	Défaut	Description
`--cnv-jitter-sd`	0	variabilité des rapports de copies
`--astro-grade4-fraction`	0	part des astrocytomes en grade 4
`--seed-base`	2026	graine de l'individu i = seed-base + i
`--jobs`	4	simulations en parallèle
`--classify`	—	classe chaque BAM après simulation
`--classify-out-dir`	cohort_classification	dossier des classifications
`--classify-jobs`	2	classifications en parallèle
`--classify-threads`	2	fils R par classification
`--num-trees`	2000	arbres de la forêt aléatoire

7.4 Classification et export des profils

Option	Défaut	Description
`bedmethyl_from_bam.py --bam --out`	obligatoire	BAM d'entrée et bedMethyl de sortie
`bedmethyl_from_bam.py --min-cov`	1	couverture minimale d'un site
`methylation_classification_local.R -s -o -i`	obligatoire	nom, dossier de sortie, bedMethyl
`… -p -a -d`	fichiers de data/	sondes, manifeste HM450, données d'entraînement
`… -t / -n`	16 / 20000	fils / nombre d'arbres
`01_export_probe_targets.R <data> <sortie>`	obligatoire	un profil représentatif par classe
`02_export_cohort_probe_targets.R <data> <sortie> <n> [graine]`	graine 2026	n profils distincts par classe
`run_full_mnp_classification.sh`	--num-trees 20000, --threads 16	--profile, --data-dir, --ref ou --standin

7.5 Rapid-CNS2, données réelles et clé USB

Option	Défaut	Description
`align_fastq_rapidcns2_en.sh -i -r -s`	obligatoire	FASTQ (fichier ou dossier), référence, échantillon
`… -o / -t / -m`	rapidcns2_input / 16 / —	dossier de sortie, fils, options minimap2
`make_usb_stick_en.sh DEVICE [-- …]`	obligatoire	formate, monte et remplit la clé (voir 6.8)
`prepare_usb_en.sh <point de montage>`	obligatoire	clé formatée en ext4
`… --engine`	apptainer si présent, sinon docker	apptainer, docker ou both
`… --no-annovar / --no-java`	—	n'embarque pas ANNOVAR / Java

Option	Défaut	Description
… --with-samples DIR / --dry-run	—	copie des BAM / simulation sans écriture
rapidcns2_gui_en.py	—	valeurs par défaut en tête du script ou variables RCNS_<CHAMP>

8. Référence des fichiers

8.1 truth_events.tsv

Colonnes : event, type, chrom, start, end, value (coordonnées 0-based, fin exclue). Types :

type	Signification	value
copy_ratio	rapport de copies d'un chromosome ou d'une région	ex. 1,5 ; 0,5 ; 9,0 ; 0,02
methylation	promoteur MGMT méthylé	0,88
background	part méthylée du fond	0,470 (gbm), 0,642 (astro, oligo)
snv	variant ponctuel	ref>alt;vaf=…
wildtype	point chaud laissé sauvage	base de référence
del	délétion de 1 pb (ATRX ou CIC)	vaf=…

8.2 Fichiers de cohorte

Fichier	Contenu
cohort_manifest.tsv	individual, profile, profile_index, seed, purity, grade4, expected_dx, mnp_file, bam, status, elapsed_s
INDnnn_<PROFILE>.log	journal de la simulation de l'individu
cohort_classification_summary.tsv	individual, profile, expected_dx, predicted_dx, correct, calibrated_score, n_features, status
cohort_probe_manifest.tsv	sortie de 02_export : profil, rang, classe, identifiant de l'échantillon réel, fichier, nombre de sondes, % méthylé

8.3 Sorties de la classification

Fichier	Contenu
<id>_calibrated_classification.tsv	Number of features, Predicted Class, Initial Score, Calibrated Score
<id>_votes.tsv	fréquence (et fréquence recalibrée) de chaque classe
<id>_rf_details.tsv	sondes recouvertes, score de Brier, sondes d'entraînement
<id>_methylation_hg38_HM450.RDS	méthylation de l'échantillon par sonde HM450
<id>_probes_for_training.csv	sondes communes à l'échantillon et au jeu de référence

8.4 bedMethyl

18 colonnes au format modkit ; une ligne par CpG (position du C sur le brin +, brin « . »). Seules les colonnes 1–3 (position), 6 (brin), 10 (couverture) et 11 (% méthylé) sont lues par la classification.

8.5 Arborescences des dossiers

Organisation du projet et des dossiers créés par les scripts, avec les noms par défaut.

Projet

Contenu de NanoPoreSeq.zip. Les scripts de simulation/ s'appellent entre eux : ils doivent rester dans le même dossier.

```
NanoPoreSeq/
├── README.md                                  présentation des dossiers
├── simulation/                                simuler, vérifier, classer
│   ├── simulate_glioma_hg38.py                simule un BAM
│   ├── verify_bam.py                          vérifie un BAM
│   ├── build_cohort.py                        simule une cohorte
│   ├── bedmethyl_from_bam.py                  BAM → bedMethyl
│   ├── methylation_classification_local.R     classe la méthylation
│   ├── 01_export_probe_targets.R              1 profil réel par classe
│   ├── 02_export_cohort_probe_targets.R       N profils réels par classe
│   ├── run_full_mnp_classification.sh         chaîne complète
│   └── simnanocns_gui_en.py                   interface Streamlit
├── rapidcns2/                                 lancer Rapid-CNS2_nf
│   ├── rapidcns2_gui_en.py                    interface Tkinter
│   └── align_fastq_rapidcns2_en.sh            FASTQ → BAM (données réelles)
├── usb/
│   ├── make_usb_stick_en.sh                   formate et crée la clé USB (tout-en-un)
│   └── prepare_usb_en.sh                      copie le contenu sur la clé
└── docs/                                      guides et rapports (FR, EN ; .docx et .pdf)
```

Échantillon simulé

Sortie de simulate_glioma_hg38.py (--out sim_gbm, profil gbm).

```
sim_gbm/                                       dossier --out
├── SIM_GBM_HG38.bam                           BAM simulé (nom = --sample-id)
├── SIM_GBM_HG38.bam.bai                       index
└── truth_events.tsv                           vérité terrain
```

Classification d'un échantillon

Sorties de 01_export_probe_targets.R et de la classification (procédure de la section 6.3).

```
probe_targets/                                 01_export_probe_targets.R
└── probe_targets_gbm.tsv, …_astro.tsv, …_oligo.tsv
classif_gbm/                                   run_full_mnp_classification.sh (-o)
├── <id>_calibrated_classification.tsv         classe prédite, scores
├── <id>_votes.tsv, <id>_rf_details.tsv
├── <id>_methylation_hg38_HM450.RDS
└── <id>_probes_for_training.csv
```

Cohorte

Sorties de build_cohort.py et de 02_export_cohort_probe_targets.R (noms par défaut).

```
cohort_bams/                                dossier --out-dir
├── cohort_manifest.tsv                     un individu par ligne, statut
├── IND001_OLIGO.log                        journal de simulation
└── IND001_OLIGO/                           un dossier par individu
    ├── IND001_OLIGO.bam (+ .bai)
    ├── IND001_OLIGO.bedmethyl.bed          avec --classify
    └── truth_events.tsv
cohort_classification/                      avec --classify (--classify-out-dir)
├── cohort_classification_summary.tsv       classe attendue / prédite
└── IND001_OLIGO/
    ├── …_calibrated_classification.tsv     classe prédite
    ├── …_votes.tsv, …_rf_details.tsv
    ├── …_methylation_hg38_HM450.RDS
    ├── …_probes_for_training.csv
    └── …_classify.log                      préfixe … = IND001_OLIGO
cohort_probes/                              02_export_cohort_probe_targets.R
├── probe_targets_gbm_01.tsv …              un fichier par individu et par classe
└── cohort_probe_manifest.tsv               échantillon réel utilisé pour chaque fichier
```

Résultats de Rapid-CNS2

Avec l'interface rapidcns2_gui_en.py, dont la sortie par défaut est results_<id> dans le dossier de lancement.

```
<dossier de lancement>/results_<id>/        sortie de Rapid-CNS2_nf
├── report/<id>_Rapid-CNS2_report_full.html rapport (et _lite.html)
├── mods/<id>.5mC.bedmethyl                 méthylation par CpG
├── mgmt/mgmt_avg_cov.txt                   couverture MGMT
└── pipeline_info/*_trace.txt               suivi des étapes
<dossier de lancement>/work/                travail Nextflow (~150 Go/échantillon)
```

Clé USB

Créée par make_usb_stick_en.sh (ou prepare_usb_en.sh sur une clé déjà formatée). Le dossier de travail de Nextflow n'est pas sur la clé : il est créé sur le disque de l'hôte (~/rapidcns2_work par défaut, modifiable par RCNS_WORKDIR).

```
/media/$USER/RAPIDCNS2/rapidcns2/
├── run.sh, run_gui.sh                      lanceurs
├── env.sh                                  environnement commun
├── README.txt
├── bin/nextflow                            Nextflow autonome
├── jre/                                    Java (sauf --no-java)
├── pipeline/                               Rapid-CNS2_nf
├── containers/sif/ ou docker/              images des outils
├── refs/                                   hg38.fa + .fai
├── annovar/                                sauf --no-annovar
├── annotsv/
├── conf/usb.config                         réglages propres à la clé
├── gui/                                    interface Rapid-CNS2
├── samples/                                BAM copiés (--with-samples)
├── results/<id>/                           résultats des analyses
└── nxf_home/                               cache Nextflow, plugins
```

9. Ressources et durées

Valeurs mesurées sur la machine de test (7 Go de mémoire, séquence de remplacement). Le génome entier n'a pas été simulé : faites un premier essai avec peu de jobs avant une grande cohorte.

Opération	Durée	Mémoire
Simulation chr10, sans sondes MNP	46 s	—
Simulation chr17, sans sondes MNP	~1 min	0,77 Go
Simulation de 6 chromosomes (~¼ du génome) avec sondes MNP	15,5 min	—
bedMethyl de ce BAM (951 528 reads)	5,6 min	—
Export 01 des profils représentatifs	23 s	—
Classification, 300 arbres, 2 fils	1,6 min	4,4 Go
Classification, 2 000 arbres	—	plus de 7 Go
Cohorte de 3 individus, chr17, avec sondes et classification (300 arbres)	8,3 min	—

- La mémoire d'une simulation croît avec la longueur du plus grand chromosome simulé (chr1 est trois fois plus long que chr17).
- Pour --classify-jobs, compter plus de 7 Go par classification à 2 000 arbres, davantage à 20 000.
- Rapid-CNS2 : prévoir ~150 Go de disque par échantillon pour le dossier de travail de Nextflow.

10. Dépannage

Message ou symptôme	Cause	Solution
`Specify either --ref hg38.fa or --standin`	aucune référence, ou les deux	donner exactement l'une des deux
`ERROR: hg38 contigs not found in …`	noms de contigs ni chr*, ni RefSeq	utiliser une FASTA hg38 UCSC ou NCBI
`ERROR: IDH1_R132H … ref base=…, expected C`	référence qui n'est pas hg38 (hg19 par exemple)	utiliser hg38 ; --skip-ref-check seulement si l'on est sûr de la version
`--chroms: unknown chromosome(s)`	nom de chromosome invalide	écrire chr1 … chr22, chrX
verify_bam.py : taux de mésappariement ~74 %	BAM --standin vérifié avec une autre graine	passer --seed <graine de la simulation>
Statut FAILED(rc=-9) dans le manifeste	processus tué, en général par manque de mémoire	réduire --jobs ; voir le .log de l'individu
`WARNING: no probe file found for …`	profils réels non exportés ou trop peu nombreux	lancer 02_export… avec n = taille du plus grand groupe
`--classify and --astro-grade4-fraction require --mnp-targets-dir`	pas de profils réels	ajouter --mnp-targets-dir
Classification R « Killed » (code 137)	mémoire insuffisante	réduire --classify-jobs ou --num-trees, ou ajouter de la mémoire
`glm.fit: algorithm did not converge`	scores parfaitement séparés lors de la recalibration	normal : la classe prédite n'est pas affectée

Message ou symptôme	Cause	Solution
Interface : « … does not know: --option »	dossier des scripts contenant une ancienne version	pointer vers NanoPoreSeq/simulation
Rapid-CNS2 : module MGMT sauté	couverture MGMT sous --minimumMgmtCov	resimuler avec un --mgmt-depth-factor plus élevé, ou baisser le seuil
Rapid-CNS2 : échec d'annotSv* ou savanaCna	attendu sur BAM simulé	custom.config (section 6.6)
Rapid-CNS2 : résultats incohérents	BAM simulé avec --standin	resimuler avec --ref
`Missing output file(s) `…_annotsv.tsv` expected by process `annotSv…` `	BAM simulé : aucun variant de structure, AnnotSV n'écrit pas de table	cocher « Simulated BAM » dans l'interface (automatique pour nos BAM), ou -c avec le fichier de la section 6.6
`AnnotSV needs … an existing path of the annotations directory`	dossier d'annotations non monté dans le conteneur Docker	interface à jour (montage automatique) ; en ligne de commande : --containerBindPaths dossier_AnnotSV,dossier_ANNOVAR
`Config parsing failed / Unexpected input: 'java'`	Nextflow ≥ 26.04 : syntaxe stricte, pas encore suivie par Rapid-CNS2_nf 3.0	export NXF_SYNTAX_PARSER=v1 (automatique dans l'interface et sur la clé)
`the stick is formatted as '…'`	clé en FAT32 ou exFAT	reformater en ext4 (section 6.8)
`… is mounted on '…': … is used by the system`	disque système ou partition montée hors de /media	vérifier le nom avec --list ; démonter la clé si elle est montée ailleurs
`… is neither USB nor removable`	boîtier non signalé comme amovible	vérifier que c'est bien la clé, puis --allow-non-removable
`… is a folder, not a mount point`	dossier qui n'est pas le point de montage d'une clé	donner le périphérique (--list) ou le dossier où la clé est montée
`no terminal to confirm`	lancé sans terminal (script, cron)	ajouter --yes
`ANNOVAR database missing or still compressed`	base ANNOVAR absente ou encore compressée	décompresser les bases ou utiliser --no-annovar
run.sh --check : ✘ usable container engine	ni Apptainer ni Docker sur l'hôte	installer l'un des deux ; forcer avec RCNS_ENGINE si besoin
`no MM:Z: tag in the FASTQ headers`	FASTQ sans méthylation	repartir des POD5 ou du BAM non aligné
`contig 'chr7' missing`	référence sans préfixe chr	utiliser une référence hg38 à préfixe chr

11. Glossaire

Terme	Définition
Échantillonnage adaptatif	séquençage Nanopore qui enrichit en temps réel les régions d'un panel (ici NPHD).
bedMethyl	table de méthylation par site CpG (format modkit).
Codélétion 1p/19q	perte combinée du bras court du chromosome 1 et du bras long du chromosome 19, propre à l'oligodendrogliome.
Classe de méthylation	catégorie tumorale du classificateur de Capper et al. (ex. GBM_RTK_II).
G-CIMP	phénotype hyperméthylé des gliomes IDH-mutés.

Terme	Définition
Point chaud	position où un variant connu est simulé ou contrôlé (TERT, IDH1, TP53).
Rapport de copies	nombre de copies relatif à un génome diploïde (1 = normal, 0,5 = perte d'une copie).
Sonde HM450	site CpG de la puce Illumina 450K, utilisé par le classificateur.
Tags MM/ML	champs du BAM qui portent la méthylation de chaque CpG d'un read.
VAF	fréquence allélique du variant : part des reads qui portent l'allèle muté.
Vérité terrain	liste exacte des événements simulés (truth_events.tsv).

12. Annexe : code de génération de la clé USB

Cette annexe rassemble tout le code nécessaire pour produire la clé : le script automatique make_usb_stick_en.sh (12.0), la suite de commandes manuelle (12.1), puis le script prepare_usb_en.sh (12.2 et 12.3), reproduits à l'identique depuis le dossier usb/.

12.0 Méthode automatique : usb/make_usb_stick_en.sh

Ce script réalise en une commande tout ce que décrit la section 12.1 (formatage compris) et appelle prepare_usb_en.sh (section 12.3). Il est reproduit ici tel quel ; la version de référence reste le fichier.

```
cd NanoPoreSeq/usb
./make_usb_stick_en.sh --list
./make_usb_stick_en.sh --dry-run /dev/sdX
./make_usb_stick_en.sh --eject /dev/sdX -- --engine apptainer --no-annovar
```

```
#!/usr/bin/env bash
# ============================================================================
# make_usb_stick_en.sh: creates a Rapid-CNS2 USB stick from start to finish
#
#   1. checks that the device is a removable USB disk, never a system disk
#   2. asks for confirmation, then ERASES it: one GPT partition formatted as ext4
#      (or only reformats the partition given, or keeps it with --keep-format)
#   3. mounts it and makes the current user its owner
#   4. runs prepare_usb_en.sh (same folder) to copy everything onto it
#   5. runs the stick's run.sh --check, then optionally ejects the stick
#
# Usage:
#   ./make_usb_stick_en.sh --list
#   ./make_usb_stick_en.sh [options] DEVICE [-- prepare_usb_en.sh options]
#
# DEVICE: whole disk (/dev/sdX: new partition table, ONE partition)
#         or partition (/dev/sdX1: only this partition is reformatted)
#         or mount folder of a stick already formatted (/media/$USER/RAPIDCNS2:
#         nothing is erased, as with --keep-format)
#
# Examples:
#   ./make_usb_stick_en.sh --list
#   ./make_usb_stick_en.sh --dry-run /dev/sdb
#   ./make_usb_stick_en.sh /dev/sdb -- --engine apptainer --with-samples ../cohort_bams
#   ./make_usb_stick_en.sh --keep-format --eject /dev/sdb1 -- --no-annovar
#
# Run it as your normal user: sudo is called only for the privileged steps.
# Needs: lsblk, findmnt, wipefs, mkfs.ext4, parted (or sgdisk), sudo;
#        udisksctl is used to mount and eject when available.
#
# Developed with the help of AI (Claude, Anthropic), under active review by Pierre Bady.
# ============================================================================
set -euo pipefail

SCRIPT_DIR=$(cd "$(dirname "$(readlink -f "$0")")" && pwd)
PREPARE="$SCRIPT_DIR/prepare_usb_en.sh"
```

```
LABEL="RAPIDCNS2"; KEEP=0; YES=0; ALLOW_NONREM=0; DRY=0; EJECT=0; LIST=0; MP_OPT=""; DEV_ARG=""
PREP_ARGS=()

usage() { sed -n '2,31p' "$0" | sed 's/^# \{0,1\}//'; cat <<EOF
Options:
  --list                  list the disks and their removable/USB status, then exit
  --label NAME            file system label (default: RAPIDCNS2, at most 16 characters)
  --keep-format           keep the existing ext4/xfs/btrfs file system of the partition
  --yes                   no interactive confirmation (the safety checks still apply)
  --allow-non-removable   accept a disk that the system does not report as removable/USB
  --mountpoint DIR        mount point if udisksctl is not available
                          (default: /media/\$USER/LABEL)
  --eject                 unmount and power off the stick at the end
  --dry-run               run the checks and show the actions, without changing anything
  -h, --help              this help
Everything after "--" is passed to prepare_usb_en.sh (e.g. --engine, --no-annovar,
--with-samples DIR, --no-java).
EOF
}

say()  { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
warn() { printf '\033[1;33mWARNING: %s\033[0m\n' "$*" >&2; }
die()  { printf '\033[1;31mERROR: %s\033[0m\n' "$*" >&2; exit 1; }
# runs a command, or only shows it with --dry-run
run()  { if [ "$DRY" = 1 ]; then printf '  [dry-run] %s\n' "$*"; else "$@"; fi; }
# name of the first partition: /dev/sdb -> /dev/sdb1, /dev/nvme0n1 or /dev/loop0 -> ...p1
part1() { case "$1" in *[0-9]) echo "${1}p1" ;; *) echo "${1}1" ;; esac; }

while [ $# -gt 0 ]; do
    case "$1" in
        --list)               LIST=1; shift ;;
        --label)              [ $# -ge 2 ] || die "--label needs a value"; LABEL=$2; shift 2 ;;
        --keep-format)        KEEP=1; shift ;;
        --yes)                YES=1; shift ;;
        --allow-non-removable) ALLOW_NONREM=1; shift ;;
        --mountpoint)         [ $# -ge 2 ] || die "--mountpoint needs a value"; MP_OPT=$2; shift 2 ;;
        --eject)              EJECT=1; shift ;;
        --dry-run)            DRY=1; shift ;;
        -h|--help)            usage; exit 0 ;;
        --)                   shift; PREP_ARGS=("$@"); break ;;
        -*)                   die "unknown option: $1 (see --help)" ;;
        *)                    [ -z "$DEV_ARG" ] || die "only one device can be given"; DEV_ARG=$1; shift ;;
    esac
done

if [ "$LIST" = 1 ]; then
    echo "Disks (TRAN=usb or RM=1: removable; MOUNTPOINT: currently mounted):"
    lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MODEL,TRAN,RM,MOUNTPOINT
    exit 0
fi
[ -n "$DEV_ARG" ] || { usage; exit 1; }
[[ "$LABEL" =~ ^[A-Za-z0-9_-]{1,16}$ ]] || die "--label: 1 to 16 characters among letters, digits, _ and -"
[ -x "$PREPARE" ] || [ -f "$PREPARE" ] || die "prepare_usb_en.sh not found next to this script ($PREPARE)"

# ------------------------------------------------------------------------------ user and sudo
if [ "$(id -u)" = 0 ]; then
    [ -z "${SUDO_USER:-}" ] || die "run this script as your normal user, not with sudo: \
it calls sudo itself, and prepare_usb_en.sh must run as you (Docker/Apptainer, Nextflow)"
    SUDO=(); OWNER=root
else
    command -v sudo >/dev/null || die "sudo is required for formatting and mounting"
    SUDO=(sudo); OWNER=$(id -un)
fi
LOG="make_usb_stick_$(date +%Y%m%d_%H%M%S).log"
[ "$DRY" = 1 ] || exec > >(tee -a "$LOG") 2>&1

# ------------------------------------------------------------------------------ checks
say "Device checks"
for tool in lsblk findmnt wipefs mkfs.ext4; do
    command -v "$tool" >/dev/null || die "$tool not found (package util-linux / e2fsprogs)"
done
DEV=$(readlink -f "$DEV_ARG")
```

```
if [ -d "$DEV" ]; then
    # a folder: accepted only if it is the mount point of a partition, which is then kept
    SRC=$(findmnt -nro SOURCE --mountpoint "$DEV" 2>/dev/null | head -1 || true)
    [ -n "$SRC" ] || die "$DEV_ARG is a folder, not a mount point: give the stick device \
(e.g. /dev/sdb, see --list) or the folder where the stick is mounted"
    echo "$DEV_ARG is the mount point of $SRC: this partition is kept, nothing is erased"
    [ -n "$MP_OPT" ] || MP_OPT=$DEV
    DEV=$(readlink -f "$SRC"); KEEP=1
fi
[ -b "$DEV" ] || die "$DEV_ARG is neither a block device nor a mount point (see --list)"
TYPE=$(lsblk -dno TYPE "$DEV")
case "$TYPE" in
    disk|loop) DISK=$DEV; PART="" ;;
    part)      DISK=/dev/$(lsblk -no PKNAME "$DEV" | head -1); PART=$DEV ;;
    *)         die "$DEV is of type '$TYPE': give a whole disk (/dev/sdX) or a partition (/dev/sdX1)" ;;
esac
[ -b "$DISK" ] || die "cannot find the disk holding $DEV"
if [ "$KEEP" = 1 ] && [ -z "$PART" ]; then
    die "--keep-format needs a partition (e.g. $(part1 "$DISK")), not a whole disk"
fi

# 1) never a disk used by the system: any mount outside /media, /run/media, /mnt and the
#    --mountpoint folder, or any swap, means the disk belongs to the system
MP_ABS=""; [ -z "$MP_OPT" ] || MP_ABS=$(readlink -m "$MP_OPT")
while read -r name mp; do
    [ -n "$mp" ] || continue
    mp=$(printf '%b' "$mp")                    # lsblk -r escapes spaces as \x20
    case "$mp" in
        /media/*|/run/media/*|/mnt|/mnt/*) continue ;;
    esac
    [ -n "$MP_ABS" ] && { [ "$mp" = "$MP_ABS" ] || [ "${mp#"$MP_ABS"/}" != "$mp" ]; } && continue
    die "$name is mounted on '$mp': $DISK is used by the system, it will not be touched"
done < <(lsblk -nrpo NAME,MOUNTPOINT "$DISK")

# 2) a removable / USB disk, unless explicitly overridden
RM=$(lsblk -dno RM "$DISK" | tr -d ' ')
TRAN=$(lsblk -dno TRAN "$DISK" | tr -d ' ')
if [ "$TRAN" != usb ] && [ "$RM" != 1 ]; then
    [ "$ALLOW_NONREM" = 1 ] || die "$DISK is neither USB nor removable (TRAN='${TRAN:-?}', RM=$RM). \
If you are sure, add --allow-non-removable"
    warn "$DISK is neither USB nor removable: accepted (--allow-non-removable)"
fi

SIZE_B=$(lsblk -dbno SIZE "$DISK" | tr -d ' ')
SIZE_H=$(lsblk -dno SIZE "$DISK" | tr -d ' ')
[ "$SIZE_B" -ge $((64 * 1024 ** 3)) ] || warn "$DISK is only $SIZE_H: a complete stick needs ~100 GB or more \
(run prepare_usb_en.sh --dry-run to know the exact size)"
lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MODEL,TRAN,MOUNTPOINT "$DISK"

if [ "$KEEP" = 1 ]; then
    # file system type: lsblk (udev), else findmnt if mounted, else blkid
    FST=$(lsblk -dno FSTYPE "$PART" | tr -d ' ')
    [ -n "$FST" ] || FST=$(findmnt -nro FSTYPE "$PART" 2>/dev/null | head -1 || true)
    [ -n "$FST" ] || FST=$("${SUDO[@]}" blkid -o value -s TYPE "$PART" 2>/dev/null || true)
    case "$FST" in ext4|ext3|xfs|btrfs|f2fs) echo "Existing file system kept: $FST" ;;
        *) die "$PART is formatted as '${FST:-none}': Nextflow needs ext4/xfs/btrfs (drop --keep-format)" ;;
    esac
    ACTION="keep $PART ($FST) and add Rapid-CNS2 to it"
elif [ -n "$PART" ]; then
    ACTION="ERASE the partition $PART and format it as ext4 (label $LABEL)"
else
    ACTION="ERASE the whole disk $DISK ($SIZE_H): one GPT partition, ext4, label $LABEL"
fi

# ------------------------------------------------------------------------- confirmation
say "Planned action: $ACTION"
if [ "$DRY" = 0 ] && [ "$KEEP" = 0 ] && [ "$YES" = 0 ]; then
    [ -t 0 ] || die "no terminal to confirm: add --yes to run without confirmation"
    printf 'ALL DATA on %s will be lost. Type the device name (%s) to confirm: ' "$DISK" "$(basename "$DISK")"
    read -r answer
    [ "$answer" = "$(basename "$DISK")" ] || die "confirmation not given: nothing was changed"
fi
```

```
[ "$DRY" = 1 ] || "${SUDO[@]}" true    # asks for the sudo password once, before the long steps

# ------------------------------------------------------------------ unmount the stick
say "Unmounting"
while read -r name mp; do
    [ -n "$mp" ] || continue
    if command -v udisksctl >/dev/null && [ "$DRY" = 0 ] && udisksctl unmount -b "$name" >/dev/null 2>&1; then
        echo "  $name unmounted"
    else
        run "${SUDO[@]}" umount "$name"
    fi
done < <(lsblk -nrpo NAME,MOUNTPOINT "$DISK" | if [ "$KEEP" = 1 ]; then grep -v "^$PART "; else cat; fi)

# ------------------------------------------------------------------ partition and format
if [ "$KEEP" = 0 ]; then
    if [ -z "$PART" ]; then
        say "New partition table on $DISK"
        while read -r child; do run "${SUDO[@]}" wipefs -aq "$child"; done < <(lsblk -nrpo NAME,TYPE "$DISK" | awk
'$2=="part"{print $1}')
        run "${SUDO[@]}" wipefs -aq "$DISK"
        if command -v parted >/dev/null; then
            run "${SUDO[@]}" parted -s "$DISK" mklabel gpt mkpart "$LABEL" ext4 1MiB 100%
        elif command -v sgdisk >/dev/null; then
            run "${SUDO[@]}" sgdisk -Z -n 1:1MiB:0 -t 1:8300 -c "1:$LABEL" "$DISK"
        else
            die "parted or sgdisk is required to partition the disk (sudo apt install parted)"
        fi
        if [ "$DRY" = 1 ]; then
            PART="<first partition of $DISK>"
        else
            command -v partprobe >/dev/null && "${SUDO[@]}" partprobe "$DISK" 2>/dev/null || true
            command -v udevadm >/dev/null && "${SUDO[@]}" udevadm settle 2>/dev/null || true
            for _ in $(seq 1 20); do
                PART=$(lsblk -nrpo NAME,TYPE "$DISK" | awk '$2=="part"{print $1; exit}')
                [ -n "$PART" ] && [ -b "$PART" ] && break
                sleep 0.5
            done
            [ -n "$PART" ] && [ -b "$PART" ] || die "the new partition of $DISK did not appear: unplug and replug the
stick, then rerun with its partition (e.g. $(part1 "$DISK"))"
        fi
    fi
    say "Formatting $PART as ext4 (label $LABEL)"
    run "${SUDO[@]}" wipefs -aq "$PART"
    run "${SUDO[@]}" mkfs.ext4 -F -q -m 0 -L "$LABEL" "$PART"
fi

# ------------------------------------------------------------------------------ mount
say "Mounting"
MP=""
if [ "$DRY" = 1 ]; then
    MP=${MP_OPT:-/media/$OWNER/$LABEL}
    run mount "$PART" "$MP"
    echo "  (udisksctl, if present, may choose another mount point)"
else
    MP=$(findmnt -nro TARGET "$PART" 2>/dev/null | head -1 || true)
    if [ -z "$MP" ] && [ -z "$MP_OPT" ] && command -v udisksctl >/dev/null; then
        out=$(udisksctl mount -b "$PART" 2>/dev/null || true)       # "Mounted /dev/sdb1 at /media/u/LABEL"
        MP=$(printf '%s' "$out" | sed -n 's/^Mounted .* at \(.*\)$/\1/p' | sed 's/\.$//')
    fi
    if [ -z "$MP" ]; then
        MP=${MP_OPT:-/media/$OWNER/$LABEL}
        "${SUDO[@]}" mkdir -p "$MP"
        "${SUDO[@]}" mount "$PART" "$MP"
    fi
    [ -d "$MP" ] || die "mounting $PART failed"
fi
[ "$DRY" = 1 ] || echo "  $PART mounted on $MP"
run "${SUDO[@]}" chown "$OWNER": "$MP"

# ------------------------------------------------------------------ content of the stick
say "Copying Rapid-CNS2 onto the stick (prepare_usb_en.sh)"
if [ "$DRY" = 1 ]; then
```

```
    run bash "$PREPARE" "${PREP_ARGS[@]}" "$MP"
    echo "  (to estimate the space needed: $PREPARE --dry-run <any ext4 mount point>)"
else
    bash "$PREPARE" "${PREP_ARGS[@]}" "$MP"
fi

say "Check of the stick"
if [ "$DRY" = 1 ]; then
    run "$MP/rapidcns2/run.sh" --check
else
    "$MP/rapidcns2/run.sh" --check || warn "some items are missing on the stick (see above)"
    sync
fi

# ------------------------------------------------------------------------------ end
if [ "$EJECT" = 1 ]; then
    say "Ejecting"
    if command -v udisksctl >/dev/null && [ "$DRY" = 0 ]; then
        udisksctl unmount -b "$PART" && udisksctl power-off -b "$DISK" || warn "eject failed: unmount it manually"
    else
        run "${SUDO[@]}" umount "$MP"
    fi
    echo "The stick can be removed."
elif [ "$DRY" = 0 ]; then
    echo
    echo "Stick ready: $MP/rapidcns2   (unmount before removing it: umount $MP)"
fi
[ "$DRY" = 1 ] && echo "--dry-run: nothing was changed." || echo "Log: $LOG"
```

12.1 Méthode manuelle : suite de commandes

À exécuter sur la machine qui contient le pipeline, la référence, ANNOVAR et AnnotSV (P3Ultra). Remplacer /dev/sdX1 par la partition de la clé indiquée par lsblk : le formatage efface la clé.

```
# 1. Identifier et formater la clé en ext4
lsblk
sudo umount /dev/sdX1
sudo mkfs.ext4 -m 0 -L RAPIDCNS2 /dev/sdX1
# 2. Rebrancher la clé, puis s'en donner la propriété
sudo chown $USER: /media/$USER/RAPIDCNS2
# 3. (Facultatif) indiquer les sources si elles diffèrent des valeurs par défaut
export PIPELINE_DIR=/export/scratch3/gitProject/Rapid-CNS2_nf
export REF_FA=/export/scratch3/data/Annotations/UCSC_HG38/hg38.fa
export ANNOVAR_DIR=/export/scratch3/data/Annotations/humandb/annovar
export ANNOTSV_DIR=/export/scratch3/gitProject/AnnotSV/share/AnnotSV
# 4. Vérifier la place nécessaire, sans rien écrire
cd NanoPoreSeq/usb
./prepare_usb_en.sh --dry-run /media/$USER/RAPIDCNS2
# 5. Construire la clé (ajouter --no-annovar si elle est remise à un tiers)
./prepare_usb_en.sh --engine apptainer --with-samples /path/to/cohort_bams \
    /media/$USER/RAPIDCNS2
# 6. Contrôler sur l'ordinateur hôte
/media/$USER/RAPIDCNS2/rapidcns2/run.sh --check
```

12.2 Ce que fait le script

1. Contrôles : système de fichiers ext4/xfs/btrfs, droits d'écriture, présence du pipeline, de la référence et de son index, d'AnnotSV, d'ANNOVAR, de nextflow, de rsync et d'un moteur de conteneurs.
2. Liste des images de conteneurs lues dans la configuration du pipeline (liste de secours si aucune n'est trouvée).
3. Estimation de la place nécessaire et comparaison avec la place libre ; arrêt ici avec --dry-run.
4. Copie de Nextflow (binaire autonome téléchargé, sinon le lanceur local) et de Java, puis du pipeline et de ses plugins Nextflow.

5. Conteneurs : images .img pour Apptainer/Singularity et/ou archives Docker.
6. Copie de la référence, d'AnnotSV, d'ANNOVAR (bases du protocole par défaut), de l'interface et des BAM demandés.
7. Écriture des lanceurs env.sh, run.sh, run_gui.sh, de conf/usb.config et de README.txt.

12.3 Script complet : usb/prepare_usb_en.sh

Le script est reproduit ici tel quel ; la version de référence reste le fichier usb/prepare_usb_en.sh. Les lanceurs de la clé (env.sh, run.sh, run_gui.sh, usb.config) sont écrits par les blocs « cat > … <<EOF » de la section « Launchers and configuration ».

```
#!/usr/bin/env bash
# Developed with the help of AI (Claude, Anthropic), under active review by Pierre Bady.
# ============================================================================
# prepare_usb_en.sh: builds a self-contained USB stick for Rapid-CNS2_nf
#
# The stick holds: the pipeline, Nextflow (+ Java), the containers, the hg38
# reference, ANNOVAR (+ the 5 databases of the default protocol), the AnnotSV
# annotations, the graphical interface and launchers (run.sh, run_gui.sh).
# Nextflow's "work" directory stays on the host's disk.
#
# The graphical interface (rapidcns2_gui_en.py) is looked for next to this script,
# then in ../rapidcns2/ (layout of the NanoPoreSeq project); or set GUI_PY.
#
# Usage: ./prepare_usb_en.sh [options] /media/$USER/MY_STICK
# Run with --dry-run first to check the sizes.
# ============================================================================
set -euo pipefail

# ----------------------------------------------------------------------------
# Sources (P3Ultra machine). Edit here or override with environment variables.
# ----------------------------------------------------------------------------
PIPELINE_DIR=${PIPELINE_DIR:-/export/scratch3/gitProject/Rapid-CNS2_nf}
REF_FA=${REF_FA:-/export/scratch3/data/Annotations/UCSC_HG38/hg38.fa}
ANNOVAR_DIR=${ANNOVAR_DIR:-/export/scratch3/data/Annotations/humandb/annovar}
ANNOVAR_DB=${ANNOVAR_DB:-$ANNOVAR_DIR/humandb}
ANNOTSV_DIR=${ANNOTSV_DIR:-/export/scratch3/gitProject/AnnotSV/share/AnnotSV}
SCRIPT_DIR=$(cd "$(dirname "$(readlink -f "$0")")" && pwd)
if [ -z "${GUI_PY:-}" ]; then
    GUI_PY=$SCRIPT_DIR/rapidcns2_gui_en.py
    [ -f "$GUI_PY" ] || GUI_PY=$SCRIPT_DIR/../rapidcns2/rapidcns2_gui_en.py
fi

# ANNOVAR databases of the default protocol (refGene,cytoBand,clinvar_20240917,avsnp151,1000g2015aug_eur)
ANNOVAR_REQUIRED=(hg38_refGene.txt hg38_refGeneMrna.fa hg38_cytoBand.txt
                  hg38_clinvar_20240917.txt hg38_avsnp151.txt hg38_EUR.sites.2015_08.txt)
ANNOVAR_OPTIONAL=(hg38_refGeneVersion.txt hg38_clinvar_20240917.txt.idx
                  hg38_avsnp151.txt.idx hg38_EUR.sites.2015_08.txt.idx)

# Fallback images if they cannot be read from the pipeline (README v3.0.x)
FALLBACK_IMAGES=(areebapatel/rapid_cns:3.0.2
                 quay.io/biocontainers/ont-modkit:0.6.4--h7f49ad2_0
                 hkubal/clair3:v2.0.2
                 quay.io/biocontainers/severus:1.7--pyhdfd78af_0
                 quay.io/biocontainers/savana:1.3.8--pyhdfd78af_0)

# ----------------------------------------------------------------------------
usage() {
    cat <<EOF
Usage: $0 [options] USB_STICK_MOUNT_POINT

Options:
  --engine apptainer|docker|both  container format on the stick
                                  (default: apptainer if installed, otherwise docker)
  --no-annovar                    do not copy ANNOVAR (stick handed to a third party:
                                  ANNOVAR may not be redistributed)
  --with-samples DIR              also copy the BAMs (+ .bai) from DIR into samples/
  --no-java                       do not bundle Java (the host must have Java >= 17)
  --dry-run                       show what would be copied and the sizes, without writing
```

```
  -h, --help                      this help

The stick must be formatted as ext4 (or xfs/btrfs), not exFAT/FAT32.
EOF
}

say()  { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
warn() { printf '\033[1;33mWARNING: %s\033[0m\n' "$*" >&2; }
die()  { printf '\033[1;31mERROR: %s\033[0m\n' "$*" >&2; exit 1; }
human() { numfmt --to=iec --suffix=B "$1" 2>/dev/null || echo "$1 B"; }
sz()    { du -sbL "$@" 2>/dev/null | awk '{s+=$1} END {print s+0}'; }
# File name Nextflow uses in its Singularity/Apptainer cache
sif_name() { local n=${1#docker://}; n=${n//\//-}; n=${n//:/-}; echo "$n.img"; }

ENGINE=""; WITH_ANNOVAR=1; SAMPLES_DIR=""; WITH_JAVA=1; DRY=0; USB=""
while [ $# -gt 0 ]; do
    case "$1" in
        --engine)       [ $# -ge 2 ] || die "--engine needs a value"; ENGINE=$2; shift 2 ;;
        --no-annovar)   WITH_ANNOVAR=0; shift ;;
        --with-samples) [ $# -ge 2 ] || die "--with-samples needs a directory"
                        SAMPLES_DIR=$(readlink -f "$2"); shift 2 ;;
        --no-java)      WITH_JAVA=0; shift ;;
        --dry-run)      DRY=1; shift ;;
        -h|--help)      usage; exit 0 ;;
        -*)             die "unknown option: $1 (see --help)" ;;
        *)              USB=$(readlink -f "$1"); shift ;;
    esac
done
[ -n "$USB" ] || { usage; exit 1; }
R="$USB/rapidcns2"

# ------------------------------------------------------------------------------
say "Checks"
# ------------------------------------------------------------------------------
[ -d "$USB" ] || die "$USB does not exist (stick not mounted?)"
FSTYPE=$(findmnt -no FSTYPE --target "$USB" 2>/dev/null || echo unknown)
case "$FSTYPE" in
    ext4|ext3|xfs|btrfs|f2fs) echo "File system: $FSTYPE ✔" ;;
    *) if [ "${RCNS_SKIP_FS_CHECK:-0}" = 1 ]; then warn "file system $FSTYPE accepted (RCNS_SKIP_FS_CHECK=1)"
       else die "the stick is formatted as '$FSTYPE'. Nextflow needs symbolic links and Unix permissions,
       and FAT32 limits files to 4 GB. Reformat it as ext4 (ERASES ALL ITS CONTENT):
         lsblk                                # identify the stick, e.g. /dev/sdX1
         sudo umount /dev/sdX1
         sudo mkfs.ext4 -m 0 -L RAPIDCNS2 /dev/sdX1
         # replug the stick, then: sudo chown \$USER: /media/\$USER/RAPIDCNS2"
       fi ;;
esac
[ "$DRY" = 1 ] || [ -w "$USB" ] || die "no write permission on $USB: sudo chown $USER: $USB"
[ -z "$SAMPLES_DIR" ] || [ -d "$SAMPLES_DIR" ] || die "--with-samples: directory not found: $SAMPLES_DIR"

[ -f "$PIPELINE_DIR/main.nf" ] || die "pipeline not found: $PIPELINE_DIR/main.nf"
[ -f "$REF_FA" ]               || die "reference not found: $REF_FA"
[ -f "$REF_FA.fai" ]           || die "missing index: samtools faidx $REF_FA"
[ -d "$ANNOTSV_DIR/Annotations_Human" ] || die "Annotations_Human missing from $ANNOTSV_DIR"
[ -f "$GUI_PY" ]               || warn "graphical interface not found ($GUI_PY): it will not be copied"
command -v nextflow >/dev/null || die "nextflow not in the PATH"
command -v rsync    >/dev/null || die "rsync required (sudo apt install rsync)"

if [ "$WITH_ANNOVAR" = 1 ]; then
    for f in "${ANNOVAR_REQUIRED[@]}"; do
        [ -f "$ANNOVAR_DB/$f" ] || die "ANNOVAR database missing or still compressed: $ANNOVAR_DB/$f"
    done
    [ -f "$ANNOVAR_DIR/table_annovar.pl" ] || die "table_annovar.pl not found in $ANNOVAR_DIR"
fi

if [ -z "$ENGINE" ]; then
    if command -v apptainer >/dev/null || command -v singularity >/dev/null; then ENGINE=apptainer
    elif command -v docker >/dev/null; then ENGINE=docker
    else die "neither Apptainer/Singularity nor Docker on this machine"; fi
fi
case "$ENGINE" in apptainer|docker|both) ;; *) die "--engine: apptainer, docker or both" ;; esac
APPT=$(command -v apptainer || command -v singularity || true)
```

```
if [ "$ENGINE" = apptainer ] || [ "$ENGINE" = both ]; then
    [ -n "$APPT" ] || die "apptainer/singularity not found to build the .sif images"
fi
if [ "$ENGINE" = docker ] || [ "$ENGINE" = both ]; then
    command -v docker >/dev/null || die "docker not found"
fi
echo "Container format: $ENGINE"

# ------------------------------------------------------------------------------
say "Container images used by the pipeline"
# ------------------------------------------------------------------------------
mapfile -t IMAGES < <(
    grep -rhoE "container[[:space:]]*=?[[:space:]]*['\"][^'\"]+['\"]" \
        "$PIPELINE_DIR"/nextflow.config "$PIPELINE_DIR"/*.nf \
        "$PIPELINE_DIR"/conf "$PIPELINE_DIR"/nextflow 2>/dev/null \
    | sed -E "s/.*['\"]([^'\"]+)['\"]$/\1/" | sed 's#^docker://##' \
    | grep -v '[$]' | grep -E '^[a-zA-Z0-9]' | sort -u )
if [ "${#IMAGES[@]}" -eq 0 ]; then
    warn "no image read from the pipeline, using the README list"
    IMAGES=("${FALLBACK_IMAGES[@]}")
fi
if grep -rqE "container[[:space:]]*=?[[:space:]]*['\"][^'\"]*[$]" \
        "$PIPELINE_DIR"/nextflow.config "$PIPELINE_DIR"/conf "$PIPELINE_DIR"/nextflow 2>/dev/null; then
    warn "some images are defined through a variable and could not be listed: check with run.sh --check"
fi
printf '  %s\n' "${IMAGES[@]}"

# ------------------------------------------------------------------------------
say "Estimating the required space"
# ------------------------------------------------------------------------------
S_PIPE=$(sz "$PIPELINE_DIR")
S_REF=$(sz "$REF_FA"*)
S_ASV=$(sz "$ANNOTSV_DIR")
S_ANV=0
if [ "$WITH_ANNOVAR" = 1 ]; then
    files=(); for f in "${ANNOVAR_REQUIRED[@]}" "${ANNOVAR_OPTIONAL[@]}"; do
        [ -f "$ANNOVAR_DB/$f" ] && files+=("$ANNOVAR_DB/$f"); done
    S_ANV=$(sz "${files[@]}" "$ANNOVAR_DIR"/*.pl)
fi
S_SMP=0; [ -n "$SAMPLES_DIR" ] && S_SMP=$(sz "$SAMPLES_DIR")
S_IMG=0
if command -v docker >/dev/null; then
    for i in "${IMAGES[@]}"; do
        s=$(docker image inspect -f '{{.Size}}' "$i" 2>/dev/null || echo 2000000000)
        S_IMG=$((S_IMG + s))
    done
else
    S_IMG=$(( ${#IMAGES[@]} * 2000000000 ))   # ~2 GB per image, estimate
fi
[ "$ENGINE" = both ] && S_IMG=$((S_IMG * 2))
S_JAVA=$(( WITH_JAVA * 400000000 ))
TOTAL=$((S_PIPE + S_REF + S_ASV + S_ANV + S_SMP + S_IMG + S_JAVA + 500000000))
AVAIL=$(df -B1 --output=avail "$USB" | tail -1 | tr -d ' ')
printf '  %-28s %10s\n' "Pipeline" "$(human "$S_PIPE")" "hg38 reference" "$(human "$S_REF")" \
       "ANNOVAR" "$(human "$S_ANV")" "AnnotSV" "$(human "$S_ASV")" \
       "Containers (approx.)" "$(human "$S_IMG")" "Java + Nextflow" "$(human "$S_JAVA")" \
       "Samples" "$(human "$S_SMP")"
printf '  %-28s %10s\n' "TOTAL (approx.)" "$(human "$TOTAL")" "Free on the stick" "$(human "$AVAIL")"
[ "$TOTAL" -lt "$AVAIL" ] || warn "free space looks insufficient"
if [ "$DRY" = 1 ]; then echo; echo "--dry-run: nothing was written."; exit 0; fi
[ "$TOTAL" -lt "$AVAIL" ] || die "not enough space on the stick"

mkdir -p "$R"/{bin,pipeline,refs,annotsv,containers/sif,containers/docker,nxf_home,conf,gui,samples,results}

# ------------------------------------------------------------------------------
say "Nextflow and Java"
# ------------------------------------------------------------------------------
NF_VER=$(nextflow -version 2>/dev/null | grep -oE 'version [0-9]+\.[0-9]+\.[0-9]+' | awk '{print $2}' | head -1 || true)
echo "Nextflow version on this machine: ${NF_VER:-unknown}"
# The self-contained binary (all dependencies included) is ideal offline. It is called
# "-dist" in recent releases and "-all" in older ones: both are tried.
NF_OK=0
```

```
if [ -n "$NF_VER" ]; then
    for suffix in dist all; do
        if curl -fsSL -o "$R/bin/nextflow.tmp" \
                "https://github.com/nextflow-io/nextflow/releases/download/v${NF_VER}/nextflow-${NF_VER}-${suffix}"; then
            mv "$R/bin/nextflow.tmp" "$R/bin/nextflow"
            echo "nextflow-${NF_VER}-${suffix} downloaded ✔"
            NF_OK=1; break
        fi
    done
fi
if [ "$NF_OK" = 0 ]; then
    rm -f "$R/bin/nextflow.tmp"
    warn "could not download the self-contained binary: copying the local launcher"
    cp "$(readlink -f "$(command -v nextflow)")" "$R/bin/nextflow"
fi
chmod +x "$R/bin/nextflow"

if [ "$WITH_JAVA" = 1 ]; then
    JAVA_BIN=$(readlink -f "$(command -v java)" 2>/dev/null || true)
    if [ -n "$JAVA_BIN" ]; then
        JHOME=$(dirname "$(dirname "$JAVA_BIN")")
        rsync -a --delete "$JHOME/" "$R/jre/"
        echo "Java copied from $JHOME ✔"
    else
        warn "java not found: the host will have to provide Java >= 17"
    fi
fi

# ------------------------------------------------------------------------------
say "Pipeline"
# ------------------------------------------------------------------------------
rsync -a --delete --exclude 'work/' --exclude '.nextflow*' --exclude 'results*/' \
      "$PIPELINE_DIR/" "$R/pipeline/"
PIPE_COMMIT=$(git -C "$PIPELINE_DIR" rev-parse --short HEAD 2>/dev/null || echo "n/a")
echo "Rapid-CNS2_nf copied (commit $PIPE_COMMIT) ✔"

# Nextflow plugins the pipeline may declare, cached on the stick
mapfile -t PLUGINS < <(grep -hoE "id[[:space:]]+['\"]nf-[^'\"]+['\"]" "$R/pipeline/nextflow.config" 2>/dev/null \
                       | sed -E "s/.*['\"](nf-[^'\"]+)['\"]/\1/" | sort -u)
(
    export NXF_HOME="$R/nxf_home"
    [ -x "$R/jre/bin/java" ] && export JAVA_HOME="$R/jre"
    "$R/bin/nextflow" -version >/dev/null 2>&1 || warn "first Nextflow launch failed"
    for p in "${PLUGINS[@]}"; do
        "$R/bin/nextflow" plugin install "$p" && echo "plugin $p ✔" || warn "plugin $p not installed"
    done
)

# ------------------------------------------------------------------------------
say "Containers"
# ------------------------------------------------------------------------------
: > "$R/containers/images.tsv"
for img in "${IMAGES[@]}"; do
    if [ "$ENGINE" = apptainer ] || [ "$ENGINE" = both ]; then
        out="$R/containers/sif/$(sif_name "$img")"
        if [ -s "$out" ]; then echo "already present: $(basename "$out")"
        else
            echo "building $(basename "$out")"
            if command -v docker >/dev/null && docker image inspect "$img" >/dev/null 2>&1; then
                "$APPT" build "$out.tmp" "docker-daemon://$img" || "$APPT" pull "$out.tmp" "docker://$img"
            else
                "$APPT" pull "$out.tmp" "docker://$img"
            fi
            mv "$out.tmp" "$out"
        fi
    fi
    if [ "$ENGINE" = docker ] || [ "$ENGINE" = both ]; then
        tgz="$(sif_name "$img")"; tgz="${tgz%.img}.tar.gz"
        if [ -s "$R/containers/docker/$tgz" ]; then echo "already present: $tgz"
        else
            docker image inspect "$img" >/dev/null 2>&1 || docker pull "$img"
            echo "exporting $img"
```

```
            docker save "$img" | gzip -1 > "$R/containers/docker/$tgz.tmp"
            mv "$R/containers/docker/$tgz.tmp" "$R/containers/docker/$tgz"
        fi
        printf '%s\t%s\n' "$img" "$tgz" >> "$R/containers/images.tsv"
    fi
done

# ------------------------------------------------------------------------------
say "Reference, AnnotSV, ANNOVAR"
# ------------------------------------------------------------------------------
REF_BASE=$(basename "$REF_FA")
rsync -aL --info=progress2 "$REF_FA" "$REF_FA".* "$R/refs/" 2>/dev/null || rsync -aL "$REF_FA"* "$R/refs/"
rsync -a --info=progress2 "$ANNOTSV_DIR/" "$R/annotsv/"
if [ "$WITH_ANNOVAR" = 1 ]; then
    mkdir -p "$R/annovar/humandb"
    rsync -a "$ANNOVAR_DIR"/*.pl "$R/annovar/"
    for f in "${ANNOVAR_REQUIRED[@]}" "${ANNOVAR_OPTIONAL[@]}"; do
        [ -f "$ANNOVAR_DB/$f" ] && rsync -aL --info=progress2 "$ANNOVAR_DB/$f" "$R/annovar/humandb/"
    done
else
    echo "ANNOVAR not copied (--no-annovar): users will have to point to their own installation"
fi

# ------------------------------------------------------------------------------
say "Graphical interface and samples"
# ------------------------------------------------------------------------------
[ -f "$GUI_PY" ] && cp "$GUI_PY" "$R/gui/rapidcns2_gui.py"
if [ -n "$SAMPLES_DIR" ]; then
    rsync -a --info=progress2 --include='*/' --include='*.bam' --include='*.bai' --exclude='*' \
          --prune-empty-dirs "$SAMPLES_DIR/" "$R/samples/"
fi

# ------------------------------------------------------------------------------
say "Launchers and configuration"
# ------------------------------------------------------------------------------
cat > "$R/env.sh" <<EOF
# shellcheck shell=bash
# env.sh: shared environment, sourced by run.sh and run_gui.sh
R=\$(cd "\$(dirname "\${BASH_SOURCE[0]}")" && pwd)
export RCNS_USB="\$R"
export PATH="\$R/bin:\$PATH"
if [ -x "\$R/jre/bin/java" ]; then export JAVA_HOME="\$R/jre"; export PATH="\$JAVA_HOME/bin:\$PATH"; fi
export NXF_HOME="\$R/nxf_home"
export NXF_OFFLINE=true
export NXF_DISABLE_CHECK_LATEST=true
# Nextflow >= 26.04 uses the strict syntax by default, not yet followed by Rapid-CNS2_nf 3.0:
# legacy parser unless already set (NXF_SYNTAX_PARSER=v2 once the pipeline is migrated)
export NXF_SYNTAX_PARSER="\${NXF_SYNTAX_PARSER:-v1}"
export NXF_SINGULARITY_CACHEDIR="\$R/containers/sif"
export NXF_APPTAINER_CACHEDIR="\$R/containers/sif"

# Container engine: can be forced with RCNS_ENGINE=singularity|docker
if [ -z "\${RCNS_ENGINE:-}" ]; then
    if { command -v apptainer || command -v singularity; } >/dev/null 2>&1 \\
       && compgen -G "\$R/containers/sif/*.img" >/dev/null; then RCNS_ENGINE=singularity
    elif command -v docker >/dev/null 2>&1 && compgen -G "\$R/containers/docker/*.tar.gz" >/dev/null; then
        RCNS_ENGINE=docker
    fi
fi
export RCNS_ENGINE="\${RCNS_ENGINE:-none}"

# Values read by the graphical interface (RCNS_ prefix + field name in upper case)
export RCNS_PIPELINE="\$R/pipeline/main.nf"
export RCNS_COHORT="\${RCNS_COHORT:-\$R/samples}"
export RCNS_RUNDIR="\${RCNS_WORKDIR:-\$HOME/rapidcns2_work}"   # work directory: host disk
export RCNS_OUTDIR="\${RCNS_OUTDIR:-\$R/results/{id}}"
export RCNS_CONFIG="\$R/conf/usb.config"
export RCNS_REF="\$R/refs/$REF_BASE"
export RCNS_ANNOVARPATH="\${RCNS_ANNOVARPATH:-\$R/annovar}"
export RCNS_ANNOVARDB="\${RCNS_ANNOVARDB:-\$R/annovar/humandb}"
export RCNS_ANNOTSVANNOT="\$R/annotsv"
export RCNS_PROFILE="local,\$RCNS_ENGINE"
```

```
export RCNS_BINDPATHS="\$R"
# wf-human-variation starts a nested Nextflow run that downloads its workflow and
# containers: impossible offline, so it is disabled by default on the stick.
export RCNS_HUMANVAR="\${RCNS_HUMANVAR:-false}"
export RCNS_MAXCPUS="\${RCNS_MAXCPUS:-\$(nproc)}"
export RCNS_MAXMEMORY="\${RCNS_MAXMEMORY:-\$(awk '/MemTotal/ {printf "%d", \$2/1048576*0.8}' /proc/meminfo)}"

rcns_check_engine() {
    if [ "\$RCNS_ENGINE" = none ]; then
        echo "ERROR: no usable container engine (Apptainer/Singularity or Docker)" >&2
        echo "       for the containers stored on the stick." >&2
        return 1
    fi
}

# Loads into Docker the stick's images that are missing on the host
rcns_load_docker() {
    [ "\$RCNS_ENGINE" = docker ] || return 0
    while IFS=\$'\\t' read -r img tgz; do
        [ -n "\$img" ] || continue
        if ! docker image inspect "\$img" >/dev/null 2>&1; then
            echo "Loading \$img into Docker…"
            gunzip -c "\$R/containers/docker/\$tgz" | docker load
        fi
    done < "\$R/containers/images.tsv"
}
EOF

cat > "$R/conf/usb.config" <<'EOF'
// usb.config: stick-specific configuration (loaded with -c by run.sh and the interface)
// The stick is mounted into the containers through --containerBindPaths (added by
// run.sh and the interface), which nextflow.config turns into -B or -v.
def usb = System.getenv('RCNS_USB')

singularity.cacheDir = "${usb}/containers/sif"
apptainer.cacheDir   = "${usb}/containers/sif"

// Steps that fail on simulated BAMs (no SV, no heterozygous SNP).
// COMMENT OUT for real data, so that errors do not go unnoticed.
process {
    withName: 'annotSv.*' { errorStrategy = 'ignore' }
    withName: 'savanaCna' { errorStrategy = 'ignore' }
}
EOF

cat > "$R/run.sh" <<'EOF'
#!/usr/bin/env bash
# run.sh: runs Rapid-CNS2 from the stick
#   ./run.sh --check                      check the installation
#   ./run.sh BAM [ID] [nextflow options]  analyse one sample
set -euo pipefail
source "$(cd "$(dirname "$(readlink -f "$0")")" && pwd)/env.sh"

if [ "${1:-}" = --check ]; then
    ok=1
    chk() { if eval "$2" >/dev/null 2>&1; then echo "  ✔ $1"; else echo "  ✘ $1"; ok=0; fi; }
    echo "Stick: $R"
    echo "Container engine: $RCNS_ENGINE"
    chk "usable container engine"          "[ '$RCNS_ENGINE' != none ]"
    chk "Java"                             "java -version"
    chk "Nextflow (offline)"               "nextflow -version"
    chk "pipeline"                         "[ -f '$RCNS_PIPELINE' ]"
    chk "reference + index"                "[ -f '$RCNS_REF' ] && [ -f '$RCNS_REF.fai' ]"
    chk "ANNOVAR (table_annovar.pl)"       "[ -f '$RCNS_ANNOVARPATH/table_annovar.pl' ]"
    chk "ANNOVAR databases"                "[ -f '$RCNS_ANNOVARDB/hg38_refGene.txt' ]"
    chk "AnnotSV"                          "[ -d '$RCNS_ANNOTSVANNOT/Annotations_Human' ]"
    chk "work directory writable"          "mkdir -p '$RCNS_RUNDIR'"
    if [ -d "$RCNS_RUNDIR" ]; then
        echo "Work directory: $RCNS_RUNDIR ($(df -h --output=avail "$RCNS_RUNDIR" | tail -1 | tr -d ' ') free; allow ~150
GB/sample)"
    fi
    echo "Resources: $RCNS_MAXCPUS CPUs, $RCNS_MAXMEMORY GB"
```

```
    echo "Samples found: $(find "$RCNS_COHORT" -maxdepth 2 -name '*.bam' 2>/dev/null | wc -l)"
    [ "$ok" = 1 ] && echo "Everything is in place." || { echo "Some items are missing."; exit 1; }
    exit 0
fi

[ $# -ge 1 ] || { sed -n '2,5p' "$0"; exit 1; }
BAM=$(readlink -f "$1"); shift
ID=$(basename "$BAM" .bam)
if [ $# -ge 1 ] && [ "${1#-}" = "$1" ]; then ID=$1; shift; fi
[ -f "$BAM" ] || { echo "BAM not found: $BAM" >&2; exit 1; }

rcns_check_engine
rcns_load_docker
mkdir -p "$RCNS_RUNDIR"; cd "$RCNS_RUNDIR"
OUT=${RCNS_OUTDIR//\{id\}/$ID}

exec nextflow run "$RCNS_PIPELINE" -ansi-log false \
    --input "$BAM" --id "$ID" --outDir "$OUT" \
    --ref "$RCNS_REF" \
    --annovarPath "$RCNS_ANNOVARPATH" --annovarDB "$RCNS_ANNOVARDB" \
    --annotsvAnnot "$RCNS_ANNOTSVANNOT" \
    --containerBindPaths "$RCNS_BINDPATHS" \
    --minimumMgmtCov 10 --mnpFlex true --runHumanVariation "$RCNS_HUMANVAR" \
    --savanaCnaArgs '--overrule_cellularity 0.8' \
    --maxCpus "$RCNS_MAXCPUS" --maxMemory "$RCNS_MAXMEMORY" \
    -profile "$RCNS_PROFILE" -c "$RCNS_CONFIG" -resume "$@"
EOF

cat > "$R/run_gui.sh" <<'EOF'
#!/usr/bin/env bash
# run_gui.sh: opens the graphical interface, preconfigured for the stick
set -euo pipefail
source "$(cd "$(dirname "$(readlink -f "$0")")" && pwd)/env.sh"
[ -f "$R/gui/rapidcns2_gui.py" ] || { echo "graphical interface missing from the stick: use ./run.sh" >&2; exit 1; }
python3 -c 'import tkinter' 2>/dev/null || { echo "tkinter missing: sudo apt install python3-tk" >&2; exit 1; }
rcns_check_engine
rcns_load_docker
mkdir -p "$RCNS_RUNDIR"; cd "$RCNS_RUNDIR"
exec python3 "$R/gui/rapidcns2_gui.py"
EOF
chmod +x "$R/run.sh" "$R/run_gui.sh"

cat > "$R/README.txt" <<EOF
Rapid-CNS2 on a USB stick
=========================
Built on $(date '+%Y-%m-%d') on $(hostname)
Pipeline: Rapid-CNS2_nf, commit $PIPE_COMMIT
Nextflow: ${NF_VER:-?}    Containers: $ENGINE
ANNOVAR included: $([ "$WITH_ANNOVAR" = 1 ] && echo yes || echo no)

Requirements on the host computer
  - Linux x86_64
  - Apptainer/Singularity (recommended) or Docker
  - python3-tk for the graphical interface
  - ~150 GB free for the work directory (default \$HOME/rapidcns2_work,
    can be changed: export RCNS_WORKDIR=/path/to/disk)

Usage
  ./run.sh --check                    check the installation on this host
  ./run_gui.sh                        graphical interface
  ./run.sh samples/X/X.bam X          command line
Results: results/<id>/   Report: results/<id>/report/<id>_Rapid-CNS2_report_full.html

Notes
  - wf-human-variation is disabled: it downloads its own workflow and
    containers, which is impossible offline (with network: export RCNS_HUMANVAR=true).
  - Offline, automatic upload to MNP-Flex is not possible;
    the file mnpflex/<id>.MNPFlex.input.bed can be uploaded by hand.
  - conf/usb.config ignores AnnotSV and SAVANA failures (simulated BAMs):
    comment these lines out for real data.
  - ANNOVAR is free for academic and non-profit use and may not be
    redistributed: do not hand a stick containing it to a third party.
```

```
EOF

sync
say "Done"
echo "Stick ready in $R ($(du -sh "$R" | cut -f1))."
echo "Check: $R/run.sh --check"
```